System and Device Programming

Project C2: Shell Implementation

OS161 Kernel Enhancement

Group 3

Faieta Luca	323770
Santurbano Davide	342269

Academic Year 2024–2025

Prof. Gianpiero Cabodi

Contents

1 Introduction

1.1 Project Overview

This project extends the OS161 operating system kernel to support **user-level process execution** through the implementation of essential process management and file system system calls. The primary objective is to enable a functional shell environment (`/bin/sh`) capable of creating, executing, and managing user programs stored on disk, with proper support for process hierarchy, standard I/O operations, and inter-process synchronization.

1.2 Objectives

The main goals of this project are:

- Implement process management system calls: `fork()`, `execv()`, `waitpid()`, `_exit()`, and `getpid()`
- Implement file system calls: `open()`, `read()`, `write()`, `lseek()`, `close()`, `dup2()`, `chdir()`, and `__getcwd()`
- Enable proper standard I/O handling (stdin, stdout, stderr)
- Support process hierarchy with parent-child relationships
- Enable running the OS161 shell to execute user programs

1.3 Development Environment

The implementation was developed on OS161 version 2.0 running on the System/161 MIPS simulator. The kernel was configured with the following options enabled:

- `OPT_WAITPID`: Process wait functionality
- `OPT_FILE`: File descriptor management
- `OPT_FORK`: Process forking capability
- `OPT_EXECV`: Program execution
- `OPT_SHELL`: Shell support features
- `OPT_SYNCH`: Synchronization primitives

2 System Architecture

2.1 Overall Design

The shell implementation follows a layered architecture consisting of:

1. **Process Management Layer**: Handles process creation, termination, and inter-process relationships
2. **File System Interface Layer**: Manages file descriptors and provides unified I/O operations
3. **Synchronization Layer**: Provides thread-safe access to shared resources

2.2 Key Design Decisions

2.2.1 Process Table Structure

We implemented a static process table with a maximum of 100 concurrent processes. Each process is assigned a unique PID through circular allocation, ensuring efficient reuse of PIDs after process termination.

2.2.2 File Descriptor Management

A two-level file descriptor architecture was adopted:

- **Per-process File Table**: Each process maintains an array of `OPEN_MAX` pointers to open file entries in its `fileTable` field
- **System-wide File Table**: A global `systemFileTable` array of size `SYSTEM_OPEN_MAX` containing actual file state information (`struct openfile`).

This design enables efficient file descriptor sharing across `fork()` operations and proper reference counting for shared file handles.

2.2.3 Synchronization Strategy

We employ a hybrid synchronization approach optimized for different use cases:

- **Spinlocks**: Used for short critical sections protecting:
  - Process structure fields (`p_lock`)
  - System-wide file table access (`systemFileTable_spinlock`)
  - Per-process file table when `OPT_SHELL` is enabled (`fileTable_spinlock`)
- **Regular Locks**: Used for operations that may sleep:
  - File offset manipulation in `struct openfile` (`of_lock`)
  - Process waiting lock when using condition variables (`p_waitlock`)
- **Condition Variables/Semaphores**: For parent-child synchronization in `waitpid()`:
  - Configurable via `USE_SEMAPHORE_FOR_WAITPID` macro
  - Semaphore implementation (`p_sem`): simpler, uses P/V operations
  - Condition variable implementation (`p_cv` with `p_waitlock`): more flexible for complex waiting patterns

3 Data Structures

3.1 Process Structure

The `struct proc` was extended to support the shell functionality:

```
struct proc {
    char *p_name;
    struct spinlock p_lock;
    unsigned p_numthreads;
    struct addrspace *p_addrspace;
    struct vnode *p_cwd;
    bool terminated;

#if OPT_WAITPID
    int p_status;
    pid_t p_pid;
    struct proc *parent;
    struct array *children;

#if USE_SEMAPHORE_FOR_WAITPID
    struct semaphore *p_sem;

#else
    struct cv *p_cv;
    struct lock *p_waitlock;

#endif
#endif

#if OPT_FILE
        struct openfile *fileTable[OPEN_MAX];

#if OPT_SHELL
        struct spinlock fileTable_spinlock;

#endif
#endif
};
```

Listing 1: Extended Process Structure

3.2 Open File Structure

Each open file is represented by:

```
struct openfile {
    struct vnode *vn;
    off_t offset;
    unsigned int countRef;
    int openflags;
    struct lock *of_lock;
};
```

Listing 2: Open File Structure

3.3 Process Table

A global process table manages all active processes:

```
static struct _processTable {
  int active;
  struct proc *proc[MAX_PROC+1];
  int last_i;
  struct spinlock lk;
} processTable;
```

Listing 3: Process Table Structure

4 System Call Implementations

4.1 Process System Calls

4.1.1 fork()

The `sys_fork()` system call creates a new process by duplicating the calling process.

Implementation steps:

1. Create a new process structure via `proc_create_runprogram()`.
2. Set the parent-child relationship.
3. Copy the parent's address space using `as_copy()`.
4. Copy the file descriptor table with reference count increments.
5. Allocate and copy the trapframe for the child.
6. Fork a new thread with `call_enter_forked_process` as entry point.
7. Set return values: Child's PID for the parent, 0 for the child (via trapframe).

```c
int sys_fork(struct trapframe *ctf, pid_t *retval) {
  struct trapframe *tf_child;
  struct proc *newp;
  int result;

  KASSERT(curthread != NULL);
  KASSERT(curproc != NULL);
  KASSERT(curproc->p_pid < MAX_PROC);

  newp = proc_create_runprogram(curproc->p_name);
  if (newp == NULL) {
    return ENOMEM;
  }

  newp->parent = curproc;

#if OPT_FILE
  proc_addChild(curproc, newp->p_pid);
#endif

  result = as_copy(curproc->p_addrspace, &(newp->p_addrspace));
  if(result) {
    proc_destroy(newp);
    return result;
  }

  proc_file_table_copy(curproc, newp);

  tf_child = kmalloc(sizeof(struct trapframe));
  if(tf_child == NULL){
    proc_destroy(newp);
    return ENOMEM;
  }
  memcpy(tf_child, ctf, sizeof(struct trapframe));

  result = thread_fork(
         curthread->t_name, newp,
         call_enter_forked_process,
         (void *)tf_child, (unsigned long)0/*unused*/);

  if (result){
    proc_destroy(newp);
    kfree(tf_child);
    return ENOMEM;
  }

  *retval = newp->p_pid;
  return 0;
}
```

Listing 4: fork() Implementation

4.1.2 execv()

The `sys_execv()` system call replaces the current process image with a new program.

Implementation Steps:

1. Copy the program path and all arguments from user space into temporary kernel buffers (`kargs`).
2. Open the executable file using `vfs_open()`.
3. Create a new address space, switch to it as the current process's address space, and activate it.
4. Load the executable into memory via `load_elf()`.
5. Define the new user stack pointer.
6. Copy the arguments strings from kernel buffers onto the new user stack (ensuring 4-byte alignment), followed by the `argv` array (ensuring 8-byte alignment).
7. Clean up the old address space and transfer control to the new entry point via `enter_new_process()`.

```
int sys_execv(const char *program, char **args) {
    char **kargs, **uargs;
    struct vnode *v;
    struct addrspace *as, *old_as;
    vaddr_t entrypoint, stackptr;
    int result, argc = 0;

    /* 1. Copy arguments from User Space to Kernel Buffers */
    // [Loop logic: counts args, allocates kargs, copies strings via copyinstr]
    // ...

    /* 2. Open the executable */
    char *kprogname = kstrdup(program);
    result = vfs_open(kprogname, O_RDONLY, 0, &v);
    if (result) return result;

    /* 3. Create and Switch Address Space */
    as = as_create();
    old_as = proc_getas();
    proc_setas(as);
    as_activate();

    /* 4. Load ELF executable */
    result = load_elf(v, &entrypoint);
    if (result) {
        proc_setas(old_as); // Restore old AS on failure
        as_activate();
        as_destroy(as);
```

```
29          return result;
30      }
31      vfs_close(v);
32
33      /* 5. Define Stack */
34      result = as_define_stack(as, &stackptr);
35
36      /* 6. Copy arguments to new User Stack */
37      uargs = kmalloc(sizeof(char *) * (argc + 1));
38
39      for (int i = argc - 1; i >= 0; i--) {
40          size_t len = strlen(kargs[i]) + 1;
41          stackptr -= len;
42          stackptr &= ~0x3; // Force 4-byte alignment
43          copyoutstr(kargs[i], (userptr_t)stackptr, len, NULL);
44          uargs[i] = (char *)stackptr;
45      }
46
47      // Push the argv array (pointers to the strings)
48      stackptr -= sizeof(char *) * (argc + 1);
49      stackptr &= ~0x7; // Force 8-byte alignment
50      copyout(uargs, (userptr_t)stackptr, sizeof(char *) * (argc + 1)
            );
51
52      /* 7. Cleanup and Enter */
53      as_destroy(old_as); // Free the old process image
54      // [Free kernel buffers: kargs, kprogname, uargs]
55
56      enter_new_process(argc, (userptr_t)stackptr, NULL, stackptr,
            entrypoint);
57
58      return EINVAL;
59  }
```

Listing 5: sys_execv implementation

4.1.3 waitpid()

The `sys_waitpid()` system call waits for a child process to terminate.

Implementation steps:

1. Ensure the PID is valid and options are zero (standard OS161 limitation).
2. Verify that the target process exists and is a direct child of the calling process.
3. Call `proc_wait()` to block the current thread until the child exits.
4. Format the returned exit code using the `_MKWAIT_EXIT` macro.
5. Safely copy the integer status to the user-space pointer provided.

```c
int sys_waitpid(pid_t pid, userptr_t statusp, int options, int *
   retval) {
    int result, s;
    struct proc *p;

    *retval = -1;

    if(pid <= 0 || pid > MAX_PROC + 1) return ESRCH;
    if(options != 0) return EINVAL;

#if OPT_WAITPID
    p = proc_search_pid(pid);
    if(p == NULL) return ESRCH;

    if(p == curproc) return EINVAL;
    if(p->parent != curproc) return ECHILD;

    s = proc_wait(p);

    // 4. Encode Exit Status
    s = _MKWAIT_EXIT(s);

    if (statusp != NULL) {
        result = copyout(&s, statusp, sizeof(int));
        if(result) return result;
    }

    *retval = pid;
    return 0;
#endif
}
```

Listing 6: sys_waitpid Implementation

4.1.4 _exit()

The `sys__exit()` system call terminates the current process.

Implementation:

1. Detach and destroy the current process's address space to free memory.
2. Save the exit status (masked to 8 bits) into the process structure for the parent to retrieve.
3. Call `proc_signal_end()` to wake up the parent process if it is waiting in `sys_waitpid()`.
4. Call `thread_exit()` to destroy the kernel thread and perform final context switching.

```
void sys__exit(int status)
{
  KASSERT(curproc != NULL);

#if OPT_WAITPID
  struct proc *p = curproc;

  struct addrspace *as = proc_getas();
  if(as != NULL) {
    proc_setas(NULL);
    as_destroy(as);
  }
  p->p_status = status & 0xff; /* just lower 8 bits returned */
  proc_signal_end(p);
#else
  /* get address space of current process and destroy */
  struct addrspace *as = proc_getas();
  as_destroy(as);
#endif
  thread_exit();

  panic("thread_exit returned (should not happen)\n");
  (void) status;
}
```

Listing 7: sys__exit Implementation

4.2 File System Calls

4.2.1 open()

The `sys_open()` system call opens a file, setting up the necessary kernel structures and returning a file descriptor.

Implementation Steps:

1. Validate pathname pointer and access mode flags (`O_RDONLY`, `O_WRONLY`, `O_RDWR`)
2. Copy pathname from userspace to kernel buffer using `copyinstr()`
3. Open file via `vfs_open()`, which returns a vnode pointer
4. Allocate entry in `systemFileTable`: find first free slot, temporarily mark as reserved (`vn = 1`) to prevent race conditions
5. Initialize `openfile` structure: create lock, set vnode, offset (0 or file size for `O_APPEND`), flags, and reference count to 1
6. Allocate file descriptor in process table: find first free slot (starting from fd 3), link to global entry, return fd number
7. Handle errors with proper cleanup (destroy locks, close vnodes, reset table entries)

```
int sys_open(userptr_t path, int openflags, mode_t mode, int *errp)
    {
    struct vnode *v;
    struct openfile *of = NULL;
    int fd, i;

    /* Validate flags */
    if ((openflags & O_ACCMODE) != O_RDONLY &&
        (openflags & O_ACCMODE) != O_WRONLY &&
        (openflags & O_ACCMODE) != O_RDWR) {
        *errp = EINVAL; return -1;
    }

    /* Copy path and Open via VFS */
    char *kbuf = kmalloc(PATH_MAX);
    copyinstr(path, kbuf, PATH_MAX, NULL);
    vfs_open(kbuf, openflags, mode, &v);
    kfree(kbuf);

    /* Allocate Global File Table Entry */
    spinlock_acquire(&systemFileTable_spinlock);
    for (i = 0; i < SYSTEM_OPEN_MAX; i++) {
        if (systemFileTable[i].vn == NULL) {
            of = &systemFileTable[i];
            of->vn = (struct vnode *)1; // Reserve slot
            break;
        }
```

```
27      }
28      spinlock_release(&systemFileTable_spinlock);
29
30      /* Initialize openfile structure */
31      of->of_lock = lock_create("openfile_lock");
32      lock_acquire(of->of_lock);
33      of->vn = v;
34      of->offset = 0;
35      of->countRef = 1;
36      of->openflags = openflags;
37
38      /* Handle O_APPEND */
39      if ((openflags & O_APPEND) == O_APPEND) {
40          struct stat statbuf;
41          VOP_STAT(of->vn, &statbuf);
42          of->offset = statbuf.st_size;
43      }
44      lock_release(of->of_lock);
45
46      /* Allocate Process File Descriptor */
47      spinlock_acquire(&curproc->fileTable_spinlock);
48      for (fd = STDERR_FILENO + 1; fd < OPEN_MAX; fd++) {
49          if (curproc->fileTable[fd] == NULL) {
50              curproc->fileTable[fd] = of;
51              spinlock_release(&curproc->fileTable_spinlock);
52              return fd;
53          }
54      }
55      spinlock_release(&curproc->fileTable_spinlock);
56
57      return -1;
58  }
```

Listing 8: sys_open Implementation

4.2.2 read() and write()

The `sys_read()` and `sys_write()` system calls transfer data between user memory and files or standard I/O streams.

Implementation Steps:

1. Validate file descriptor range (0 to `OPEN_MAX`) and check buffer pointer is not NULL
2. For standard I/O streams (`stdin`, `stdout`, `stderr`): use `getch()` for reading and `putch()` for writing, with kernel buffer and `copyin/copyout` for userspace transfer
3. For regular files (when `OPT_FILE` is enabled): delegate to helper functions `file_read()` and `file_write()`
4. In helper functions: retrieve `openfile` structure from process table, validate file permissions (`O_WRONLY` for reads, `O_RDONLY` for writes), setup `uio` structure, perform I/O via `VOP_READ()` or `VOP_WRITE()`, and update file offset atomically

5. Return number of bytes transferred in `retval`

Design Considerations: The actual file I/O logic is encapsulated in two helper functions: `file_read()` handles reading from regular files, while `file_write()` handles writing. Both helpers validate the file descriptor, check access permissions based on the `openflags` field, and perform the actual data transfer through the VFS layer. The `uio` structure is configured for direct userspace access (`UIO_USERISPACE`), allowing data to be transferred directly between the file and userspace without an intermediate kernel buffer.

```
int
sys_read(int fd, userptr_t buf_ptr, size_t size, int *retval)
{
  int i, result;
  char *kbuf;
  *retval = -1;

  if(fd < 0 || fd >= OPEN_MAX)
    return EBADF;

  if(buf_ptr == NULL)
    return EFAULT;

  if (fd!=STDIN_FILENO) {
#if OPT_FILE
  return file_read(fd, buf_ptr, size, retval);
#else
  kprintf("sys_read supported only to stdin\n");
  return EINVAL;
#endif
  }

  kbuf = (char *)kmalloc(size);
  if (kbuf == NULL) {
    return ENOMEM;
  }

  for (i=0; i<(int)size; i++) {
    kbuf[i] = getch();
    if (kbuf[i] < 0) { /* EOF or error encountered */
      result = copyout(kbuf, buf_ptr, i);
      kfree(kbuf);
      if(result) /* copyout failed */
        return result;
      *retval = i;
      return 0;
    }
  }

  result = copyout(kbuf, buf_ptr, size);
  kfree(kbuf);

```

```
  if(result)
    return result;

  *retval = (int)size;
  return 0;
}
```

Listing 9: sys_read Implementation

```
int
sys_write(int fd, userptr_t buf_ptr, size_t size, int *retval)
{
  int i, result;
  char *kbuf;

  *retval = -1;

  if(fd < 0 || fd >= OPEN_MAX)
    return EBADF;

  if(buf_ptr == NULL)
    return EFAULT;

  if (fd!=STDOUT_FILENO && fd!=STDERR_FILENO) {
#if OPT_FILE
    return file_write(fd, buf_ptr, size, retval);
#else
    kprintf("sys_write supported only to stdout\n");
    return -1;
#endif
  }

  kbuf = (char *)kmalloc(size);
  if (kbuf == NULL) {
    return ENOMEM;
  }

  result = copyin(buf_ptr, kbuf, size);
  if (result) {
    kfree(kbuf);
    return result;
  }

  for (i=0; i<(int)size; i++) {
    putch(kbuf[i]);
  }

  kfree(kbuf);
  *retval = (int)size;
  return 0;
}
```

Listing 10: sys_write Implementation

4.2.3 lseek()

The `sys_lseek()` system call repositions the file offset for an open file descriptor.

Implementation Steps:

1. Validate file descriptor, retrieve `openfile` structure, and check if vnode is seekable
2. Calculate new offset based on `whence`: `SEEK_SET` uses `pos` directly, `SEEK_CUR` adds to current offset, `SEEK_END` adds to file size from `VOP_STAT()`
3. Validate new offset is non-negative and update `of->offset` atomically under lock

```
int sys_lseek(int fd, off_t pos, int whence, int32_t *retval,
   int32_t *retval2) {
    struct openfile *of;
    struct vnode *vn;
    off_t new_offset;

    /* ... Validation: fd range, openfile, vnode ... */

    if(!VOP_ISSEEKABLE(vn)) return ESPIPE;

    switch (whence) { // Calculate new offset based on whence
        case SEEK_SET:
            new_offset = pos;
            break;
        case SEEK_CUR:
            lock_acquire(of->of_lock);
            new_offset = of->offset + pos;
            lock_release(of->of_lock);
            break;
        case SEEK_END:
            new_offset = statbuf.st_size + pos;
            break;
        default:
            return EINVAL;
    }

    if(new_offset < 0)
        return EINVAL;

    lock_acquire(of->of_lock);
    of->offset = new_offset;     // Update offset atomically
    lock_release(of->of_lock);

    *retval = (int32_t)(new_offset >> 32);
    *retval2 = (int32_t)(new_offset & 0xFFFFFFFF);

    return 0;
}
```

Listing 11: sys_lseek Logic (simplified)

4.2.4 dup2()

The `sys_dup2()` system call duplicates an existing file descriptor to a specified target descriptor number.

Implementation Steps:

1. Validate both `oldfd` and `newfd` are within valid range [0, `OPEN_MAX`)
2. If `oldfd == newfd`, return immediately with success (no operation needed)
3. Close any existing file at `newfd`: decrement reference count, and if it reaches zero, close vnode and destroy lock
4. Duplicate the file descriptor: point `newfd` to the same `openfile` structure as `oldfd` and increment reference count

```
int sys_dup2(int oldfd, int newfd, int *retval) {
    struct openfile *old_of, *new_of;

    /* ... Validate oldfd and newfd ranges ... */

    spinlock_acquire(&curproc->fileTable_spinlock);

    old_of = curproc->fileTable[oldfd];
    /* ... Verify old_of is not NULL ... */

    /* Special case: same descriptor */
    if (oldfd == newfd) {
        *retval = newfd;
        spinlock_release(&curproc->fileTable_spinlock);
        return 0;
    }

    /* Get existing entry at newfd (may be NULL) */
    new_of = curproc->fileTable[newfd];

    /* Point newfd to oldfd's openfile */
    curproc->fileTable[newfd] = old_of;
    spinlock_release(&curproc->fileTable_spinlock);

    /* Increment reference count for shared file */
    lock_acquire(old_of->of_lock);
    old_of->countRef++;
    lock_release(old_of->of_lock);

    /* Clean up previous file at newfd if it existed */
    if (new_of != NULL) {
        lock_acquire(new_of->of_lock);
        new_of->countRef--;

        if (new_of->countRef == 0) {
```

```
            /* Last reference - close file */
            /* ... vfs_close and lock_destroy ... */
        } else {
            lock_release(new_of->of_lock);
        }
    }

    *retval = newfd;
    return 0;
}
```

Listing 12: sys_dup2 Implementation (simplified)

4.2.5 close()

The `sys_close()` system call closes an open file descriptor and releases associated resources.

Implementation Steps:

1. Validate file descriptor is in range [0, `OPEN_MAX`)
2. Remove entry from process file table and retrieve `openfile` structure
3. Decrement reference count: if count reaches zero, close vnode via `vfs_close()` and destroy lock; otherwise, just release lock

```
int sys_close(int fd) {
    struct openfile *of;
    struct vnode *vn;
    struct lock *lock_to_destroy = NULL;

    /* ... Validate fd range ... */

    spinlock_acquire(&curproc->fileTable_spinlock);
    of = curproc->fileTable[fd];
    /* ... Check of is not NULL ... */

    /* Remove from process table */
    curproc->fileTable[fd] = NULL;
    spinlock_release(&curproc->fileTable_spinlock);

    lock_acquire(of->of_lock);
    /* ... Verify vnode is valid ... */

    vn = of->vn;
    of->countRef--;

    /* Last reference - cleanup required */
    if (of->countRef == 0) {
        of->vn = NULL;
        lock_to_destroy = of->of_lock;
```

```
26          lock_release(of->of_lock);
27
28          vfs_close(vn);
29          lock_destroy(lock_to_destroy);
30      } else {
31          /* Other processes still using this file */
32          lock_release(of->of_lock);
33      }
34
35      return 0;
36  }
```

Listing 13: sys_close Implementation (simplified)

4.2.6 chdir()

The `sys_chdir()` system call changes the current working directory of the calling process.

Implementation Steps:

1. Allocate kernel buffer of size `PATH_MAX`
2. Copy pathname from userspace to kernel buffer using `copyinstr()`
3. Call `vfs_chdir()` to change directory (validates path exists and is a directory)
4. Free kernel buffer and return result

```
1   int sys_chdir(const_userptr_t pathname) {
2       char *path;
3       int result;
4
5       /* Allocate kernel buffer */
6       path = (char *)kmalloc(PATH_MAX);
7       if (path == NULL)
8           return ENOMEM;
9
10      /* Copy pathname from userspace */
11      result = copyinstr(pathname, path, PATH_MAX, NULL);
12      if (result) {
13          kfree(path);
14          return result;
15      }
16
17      /* Change directory via VFS */
18      result = vfs_chdir(path);
19      kfree(path);
20
21      return result;
22  }
```

Listing 14: sys_chdir Implementation (simplified)

4.2.7 __getcwd()

The `sys___getcwd()` system call retrieves the absolute pathname of the current working directory.

Implementation Steps:

1. Verify process has a current working directory (`p_cwd` is not NULL)
2. Setup `uio` structure for userspace output: configure with `UIO_USERSPACE` flag to write directly to user buffer
3. Call `vfs_getcwd()` which traverses directory hierarchy and constructs absolute path
4. Calculate actual path length from residual count and return in `retval`

```
int sys___getcwd(userptr_t buf, size_t buflen, int *retval) {
    struct uio u_uio;
    struct iovec u_iov;
    int result;

    /* Check current directory exists */
    if (curproc->p_cwd == NULL)
        return ENOENT;

    /* Setup UIO for userspace write */
    u_iov.iov_ubase = buf;
    u_iov.iov_len = buflen;

    u_uio.uio_iov = &u_iov;
    u_uio.uio_iovcnt = 1;
    u_uio.uio_offset = 0;
    u_uio.uio_resid = buflen;
    u_uio.uio_rw = UIO_READ;
    u_uio.uio_segflg = UIO_USERSPACE;
    u_uio.uio_space = curproc->p_addrspace;

    /* Get current directory path via VFS */
    result = vfs_getcwd(&u_uio);
    if (result)
        return result;

    /* Calculate actual path length (including null terminator) */
    *retval = buflen - u_uio.uio_resid;

    return 0;
}
```

Listing 15: sys___getcwd Implementation (simplified)

5 Console Initialization

Standard file descriptors (stdin, stdout, stderr) must be initialized before running user programs:

```
int console_initialization(const char *lockname, struct proc *p,
                           int fd, int openflags) {
    struct vnode *console_vn;
    struct openfile *of;
    char console_path[] = "con:";

    /* Open console device */
    vfs_open(console_path, openflags, 0, &console_vn);

    /* Allocate system file table entry */
    /* ... */

    /* Initialize and add to process file table */
    of->vn = console_vn;
    of->offset = 0;
    of->countRef = 1;
    of->openflags = openflags;

    p->fileTable[fd] = of;
    return 0;
}
```

Listing 16: Console Initialization

Called from `runprogram()` for stdin (O_RDONLY), stdout (O_WRONLY), and stderr (O_WRONLY).

6 Synchronization Mechanisms

The kernel employs different synchronization primitives optimized for specific operation characteristics: spinlocks for brief critical sections, regular locks for operations requiring sleep capability, and condition variables or semaphores for process waiting.

6.1 Lock Implementation

Locks use wait channels for efficient thread blocking when contention occurs:

```
void lock_acquire(struct lock *lock) {
    spinlock_acquire(&lock->lk_lock);
    while (lock->lk_owner != NULL) {
        wchan_sleep(lock->lk_wchan, &lock->lk_lock);
    }
    lock->lk_owner = curthread;
    spinlock_release(&lock->lk_lock);
}

void lock_release(struct lock *lock) {
    spinlock_acquire(&lock->lk_lock);
    lock->lk_owner = NULL;
    wchan_wakeone(lock->lk_wchan, &lock->lk_lock);
    spinlock_release(&lock->lk_lock);
}
```

Listing 17: Lock Acquire/Release

The `lk_owner` field enables ownership tracking, preventing incorrect release attempts and supporting condition variable operations.

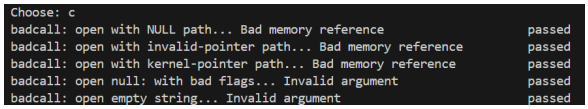
7 Testing

7.1 Test Methodology

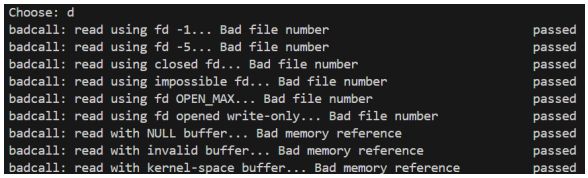
The implementation was validated through a comprehensive testing strategy covering individual system calls, shell integration, and process management. Testing was performed using the OS161 test suite, which includes unit tests for error handling (badcall tests) and integration tests for complete workflows.

7.2 System Call Validation Tests

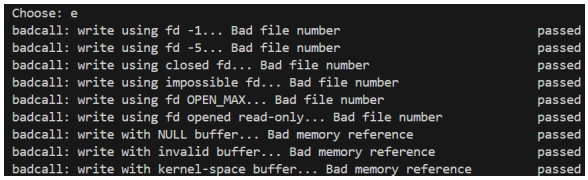
The `badcall` test suite validates error handling and boundary conditions for all implemented system calls. Each system call is tested with invalid parameters to ensure proper error code returns.



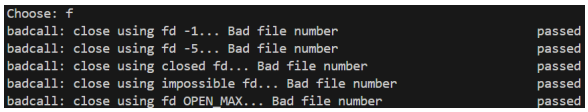
(a) `open()` bad call tests



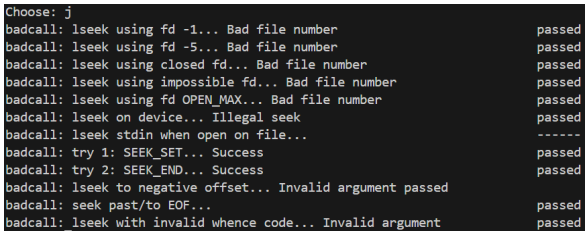
(b) `read()` bad call tests



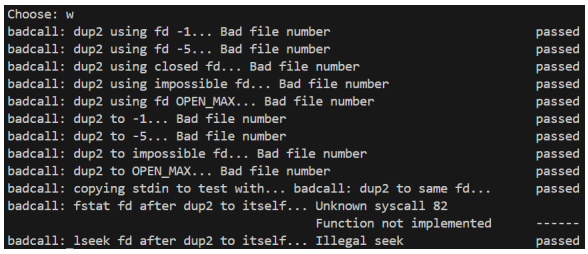
(c) `write()` bad call tests



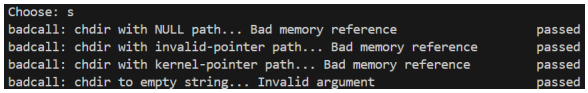
(d) `close()` bad call tests



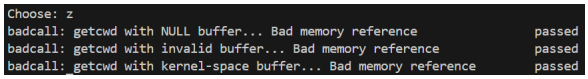
(e) `lseek()` bad call tests



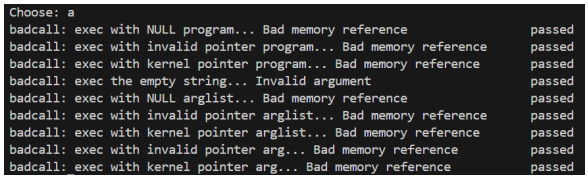
(f) `dup2()` bad call tests



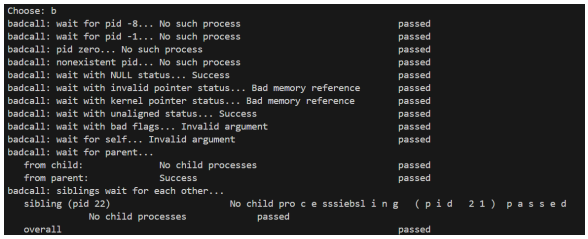
(g) `chdir()` bad call tests



(h) `__getcwd()` bad call tests



(i) `execv()` bad call tests



(j) `waitpid()` bad call tests

Figure 1: Badcall tests results

All bad call tests validate proper error handling for:

- Invalid file descriptors (negative, out of range, not open)
- NULL or invalid user pointers
- Invalid flags and parameters
- Permission violations
- Resource exhaustion scenarios

7.3 Shell Integration Tests

The shell was tested with standard OS161 utilities to verify complete workflow functionality.

```
OS/161 kernel [? for menu]: s
OS/161$ pwd
emu0:
/bin/sh: subprocess time: 0.093855490 seconds
OS/161$ cat testfile
Twiddle dee dee, Twiddle dum dum.......
/bin/sh: subprocess time: 0.189345084 seconds
OS/161$ cp testfile testfilecopy
/bin/sh: subprocess time: 0.133048231 seconds
OS/161$ cat testfilecopy
Twiddle dee dee, Twiddle dum dum.......
/bin/sh: subprocess time: 0.189378667 seconds
OS/161$ true
/bin/sh: subprocess time: 0.077529538 seconds
OS/161$ false
/bin/sh: subprocess time: 0.077575471 seconds
Exit 1
OS/161$ testbin/forktest
testbin/forktest: Starting. Expect this many:
|---------------------------|
AABBBBCCCCCCCCDDDDDDDDDDDDDDDD
testbin/forktest: Complete.
/bin/sh: subprocess time: 0.959878131 seconds
OS/161$ testbin/filetest testfilecopy
Passed filetest.
/bin/sh: subprocess time: 0.154375571 seconds
```

Listing 18: Shell basic operations

All integration tests completed successfully, demonstrating that the implemented system calls work correctly in realistic usage scenarios and support full shell functionality.

8 Workload Distribution

Team Member	Responsibilities
Davide Santurbano	Process syscalls (fork, execv, waitpid, _exit, getpid), process table management, trapframe handling, console initialization
Luca Faieta	File syscalls (open, read, write, close, lseek, dup2), chdir, getcwd, testing

Table 1: Team Workload Distribution

A Source File Summary

File	Description
`kern/proc/proc.c`	Process management, waitpid support
`kern/syscall/file_syscalls.c`	File system call implementations
`kern/syscall/proc_syscalls.c`	Process system call implementations
`kern/syscall/runprogram.c`	Program loading and execution
`kern/syscall/loadelf.c`	ELF executable loading
`kern/thread/synch.c`	Synchronization primitives
`kern/include/proc.h`	Process structure definitions
`kern/include/syscall.h`	System call prototypes

Table 2: Modified Source Files